A3-PS002-TC001

Purpose-Based Consent Recording & Withdrawal Prototype

Sujal Sharma
Anushka Parwal

National Forensic Science University, Delhi

AIORI Hackathon 3

October 5, 2026

Category: Cyber Security • Synthetic data only • Not a legal-compliance certification

Problem Description

Clear outline of the problem: recording consent is not the same as *enforcing* it.

- **Recording ≠ enforcement.** A consent checkbox or database flag does not stop a downstream request from receiving data.
- **Purpose mix-up.** Consent for analytics (P1) must never authorise recommendations (P2) or marketing (P3).
- **Stale withdrawal.** A cached or previously recorded grant can keep working after the user has withdrawn.
- **Weak evidence.** A current status alone cannot show *which notice* was accepted, or *when* a request was allowed or denied.
- **Silent notice drift.** An old grant must not be treated as acceptance of a materially changed notice.

Our contribution – ConsentDemo. A small, backend-first service in which the recorded consent decision **is** the access-control decision: one synthetic service, three purposes, a mock downstream API that fails closed, a tamper-evident audit trail, and 25+ automated tests (12 mandated + extras).

Security property proven. A request is allowed **only if** the consumer is authorised **and** the user holds active consent for **that exact purpose** under the **current notice policy**. Missing, withdrawn, mismatched, stale or invalid consent ⇒ **DENY**.

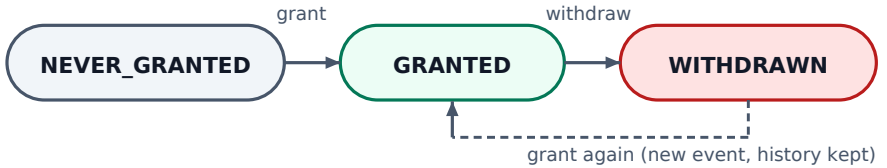
Background

Concepts needed to follow the problem and the proposed design.

- **Purpose limitation.** Personal data may be used only for the purposes the person agreed to; consent is therefore *per purpose* [1][2].
- **Withdrawal must bite.** Withdrawing must be as easy as granting, and processing must stop within a reasonable time [1][2].
- **Notice evidence.** A valid consent record ties the choice to the exact notice text shown (version + hash) [3][4].
- **Enforcement gap.** In multi-service systems, caches, missed events and unclear ownership let old permissions survive [5].

ID	Purpose	Synthetic data
P1	Service improvement	Usage events
P2	Personalised recommendations	Preference profile
P3	Marketing communications	Contact preference

Effective consent state is *derived* from the append-only event history – never stored as a lone flag:



Derived overlay: a GRANTED record whose notice is superseded by a *material* version is evaluated as **RENEWAL_REQUIRED** → DENY.

Why this matters for a hackathon prototype: the hard part is not storing a choice but guaranteeing that **every** data-sharing path consults the **current** choice, and that the evidence cannot be silently rewritten.

Literature & RFC Review

Only the most relevant standards, laws and studies – and what we take from each.

Source	What it says	How ConsentDemo uses it
GDPR Art. 6, 7(3) [1]	Consent must be specific; withdrawal as easy as giving; evidence of consent	Per-purpose grants; withdraw endpoint; stored grant evidence
India DPDP Act 2023, §6 [2]	Free, specific, informed consent; right to withdraw; processing to cease on withdrawal	Withdrawal immediately flips effective state → next request denied
ISO/IEC 29184:2020 [3]	Content and handling of online privacy notices and consent	Notice registry with canonical text and explicit versions
ISO/IEC TS 27560:2023; Kantara Consent Receipt [4]	Standard structure of a consent record / receipt	Event fields: event id, user, purpose, notice version + hash, timestamp
NIST SP 800-162 (ABAC) [5]	Policy Enforcement Point vs Policy Decision Point; attribute-based decisions	PEP (gateway) calls PDP (DecisionEngine); deny-by-default
OWASP API Security Top 10 (2023) [6]	API1 Broken Object-Level Authorization; API5 Function-Level Authorization	Tests: other user's consent, unregistered consumer, wrong purpose
RFC 8785, 6234 (SHA-2), 9457 [7]	JSON canonicalisation; SHA-256; machine-readable HTTP errors	Canonical notice hashing; hash-linked audit; problem+json reason codes
Schneier & Kelsey 1998 [8]	Forward-secure, tamper-evident logs using hash chains	Audit log chained by SHA-256; /audit/verify detects edits

Gap we address: standards describe *what* to record; they do not show that a recorded withdrawal actually blocks a live data request. Our contribution is a small, testable enforcement pattern that links the two.

Proposed Solution

Consent decision = access-control decision, enforced server-side, fail-closed, and fully evidenced.

Decision pipeline (POST /share/request)

1. **Authenticate consumer**
   hashed API key; registered & active?
2. **Bind purpose**
   purpose exists AND is on the consumer's allow-list
3. **Resolve consent**
   fold user's events → GRANTED / WITHDRAWN / NEVER
4. **Check notice policy**
   verify stored hash; material change ⇒ renewal
5. **Decide + audit atomically**
   write ALLOW/DENY record in the same DB transaction
6. **Release minimum data**
   only fields permitted for that purpose; else problem+json

Design principles

- **Deny by default.** Any missing, unknown or invalid input ⇒ 403; the first failing check wins and is logged with a reason code.
- **Events are the source of truth.** Append-only consent events; state is computed, never overwritten.
- **Evidence-first.** Every grant stores notice version + SHA-256; every decision stores request ID, purpose, reason.
- **No decision caching.** Each request reads current state – removes the stale-grant failure mode.
- **Backend decides** – never the browser or the consumer.

Scope & honest limits: one synthetic service, 3 purposes, 1 mock consumer; only requests passing through the gateway are enforced; withdrawal does not erase data already copied elsewhere; notice-change handling is a documented prototype policy – not legal advice or a GDPR/DPDP certification.

Optimization Proposed

Baseline MVP (team proposal) vs. hardened design – same 6–8 h scope, stronger guarantees.

Area	Before (baseline MVP)	After (proposed optimisation)	Benefit
Check → release race	Documented as a limitation	Single **BEGIN IMMEDIATE** SQLite transaction: read state, write audit, release data	No allow after a committed withdrawal
Audit log	Plain event table	Append-only + **SHA-256 hash chain**; triggers block UPDATE/DELETE; **/audit/verify**	Tamper-evident evidence
Notice change	Single “renewal required” rule	Per-notice **materiality flag**; stored hash re-verified at decision time	Only material edits force renewal; detects altered notice text
Consumer control	“Is registered?” check	Hashed API key + per-consumer **purpose allow-list**	Purpose binding at consumer level
Data released	Whole synthetic record	**Field-level minimisation** per purpose	Limits blast radius of any bug
Consent state	Mutable status field	State **derived by folding events**; 0/1/-1 only as a view	Single source of truth; full history
Errors & idempotency	Ad-hoc messages	RFC 9457 problem+json with reason codes; **request-ID idempotency**	Clear, testable denials; safe retries
Testing	12 scenarios	12 mandated + **concurrency, tamper, idempotency, property-based** (≈25); pytest in CI	Evidence that it fails closed

Net effect: the same small scope, but the three properties judges can verify live – **withdrawal is atomic**, **denials are explained**, and **evidence cannot be edited unnoticed** – are now guaranteed by design rather than documented as limitations.

Solution Architecture & Design

Visual: UML component view – interface, domain and persistence layers (PEP/PDP highlighted).

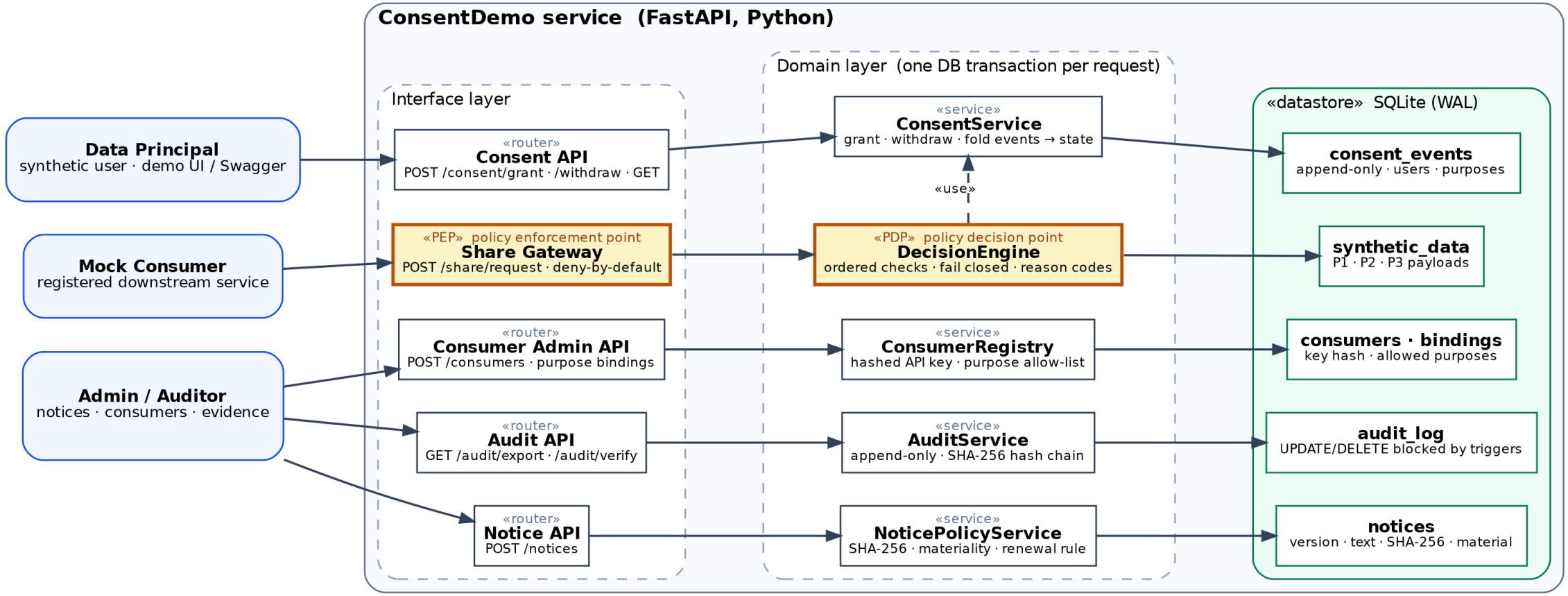
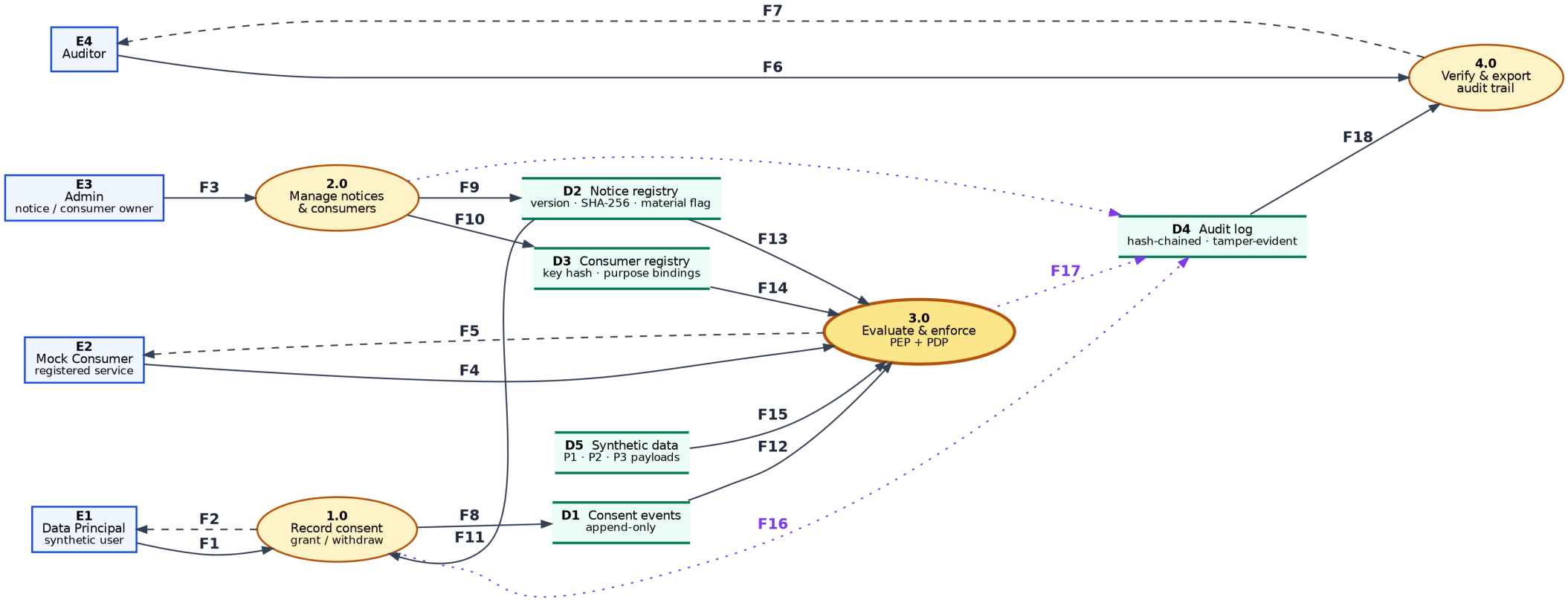


Figure: System architecture (UML component diagram). The DecisionEngine (PDP) composes the Consent, Notice, Consumer and Audit services inside one database transaction; the Share Gateway (PEP) is the only route to protected data.

Boundaries: Trust – consumers are untrusted until their key and purpose binding are verified. **Transaction** – decision, audit entry and data release commit together. **Evidence** – audit_log is append-only and hash-chained, so edits are detectable.

Data Flow Diagram

Level-1 view of the architecture: processes, data stores and external actors, with every flow numbered.

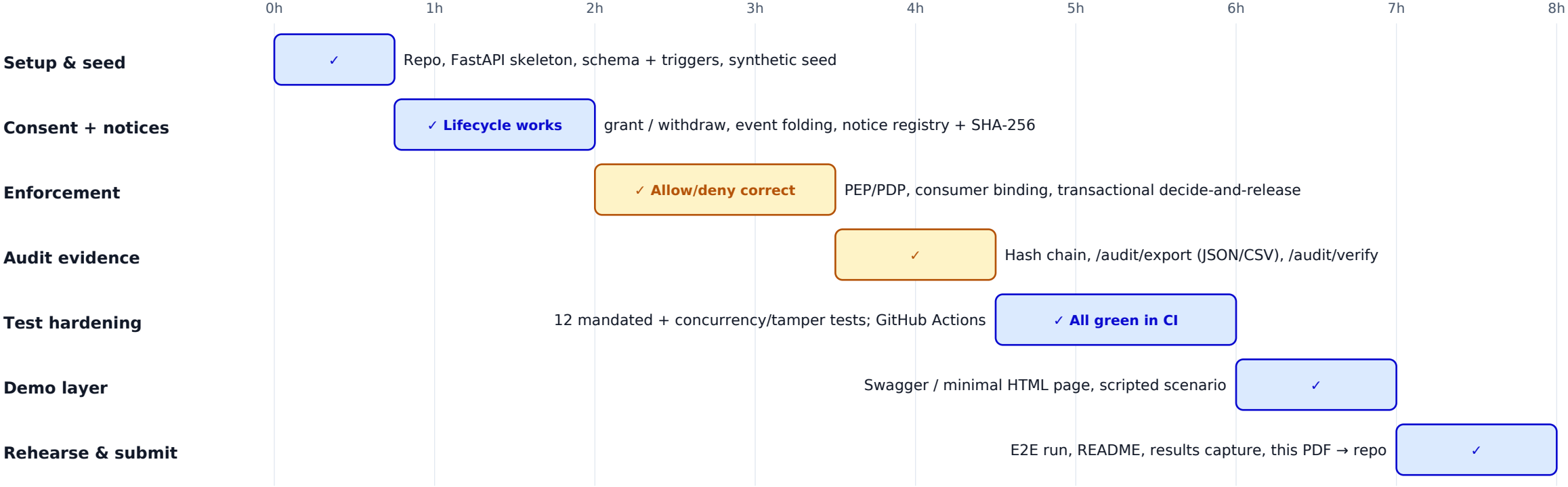


F1 E1→P1: grant / withdraw (user, purpose, notice ver.)
F2 P1→E1: consent receipt (event id, state, hash)
F3 E3→P2: notice text, materiality, consumer + purposes
F4 E2→P3: share request (key, user, purpose, req id)
F5 P3→E2: ALLOW + minimised payload or DENY + reason code
F6 E4→P4: export / verify
F7 P4→E4: JSON / CSV + chain status
F8 P1→D1: append event
F9 P2→D2: store version + hash
F10 P2→D3: register consumer
F11 D2→P1: validate notice
F12 D1→P3: effective state
F13 D2→P3: renewal policy
F14 D3→P3: authorised purposes
F15 D5→P3: purpose-scoped fields
F16 P1→D4: audit
F17 P3→D4: every decision
F18 D4→P4: read chain

F16 = every process (1.0, 2.0, 3.0) appends its event to D4; dashed = response flow; dotted = audit append.

Deployment Timeline

Milestones and methodology for the 6–8 hour build window (backend-first, test-first on enforcement).



Methodology

- **Vertical slices**: each phase ends with a runnable, demonstrable behaviour.
- **Test-first for enforcement**: negative cases written before the PDP.
- **Feature branches + CI**: pytest on every push; results captured for the demo.

Beyond the MVP (roadmap)

- Event propagation to consumers with acknowledgements (outbox pattern).
- Connected-service graph; Merkle-anchored audit checkpoints.
- Distributed enforcement with versioned consent epochs.

Bibliography I

[1] Regulation (EU) 2016/679 (General Data Protection Regulation), Arts. 5–7. *Official Journal of the EU* L119, 2016.

[2] Government of India. “The Digital Personal Data Protection Act, 2023”, Sec. 6 (Consent). Act No. 22 of 2023.

[3] ISO/IEC 29184:2020. *Information technology – Online privacy notices and consent*. ISO, 2020.

[4] ISO/IEC TS 27560:2023. *Privacy technologies – Consent record information structure*; and Kantara Initiative, *Consent Receipt Specification v1.1.0*, 2018.

[5] V. C. Hu et al. “Guide to Attribute Based Access Control (ABAC) Definition and Considerations”. *NIST SP 800-162*, 2014.

[6] OWASP Foundation. “OWASP API Security Top 10 – 2023”. owasp.org/API-Security.

[7] A. Rundgren et al. “JSON Canonicalization Scheme (JCS)”, *RFC 8785*, 2020; D. Eastlake, T. Hansen. “US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)”, *RFC 6234*, 2011; M. Nottingham, E. Wilde, S. Dalal. “Problem Details for HTTP APIs”, *RFC 9457*, 2023.

[8] B. Schneier, J. Kelsey. “Cryptographic Support for Secure Logs on Untrusted Machines”. *USENIX Security Symposium*, 1998.

Thank You for Your Attention!

A3-PS002-TC001 • Purpose-Based Consent Recording & Withdrawal Prototype

Sujal Sharma & Anushka Parwal • National Forensic Science University, Delhi